

Sistema de biblioteca com MongoDB Atlas

Parte 1: Modelagem do banco de dados

Foram criadas seis coleções com validação de schema (`$jsonSchema`).

Comandos:

```
db.createCollection("usuarios", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nome", "email", "curso", "cidade", "dataC
adaastro", "ativo"],
      properties: {
        nome: { bsonType: "string" },
        email: { bsonType: "string" },
        curso: { bsonType: "string" },
        cidade: { bsonType: "string" },
        dataCadastro: { bsonType: "date" },
        ativo: { bsonType: "bool" },
      },
    },
  },
});

db.createCollection("livros", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [
        "isbn",
        "titulo",
        "autor",
        "editora",
        "ano",
```

```

        "categoria",
        "palavrasChave",
        "quantidade",
    ],
    properties: {
        isbn: { bsonType: "string" },
        titulo: { bsonType: "string" },
        autor: { bsonType: "string" },
        editora: { bsonType: "string" },
        ano: { bsonType: "number" },
        categoria: { bsonType: "string" },
        palavrasChave: { bsonType: "array", items: { bsonType: "string" } },
        quantidade: { bsonType: "number" },
    },
},
},
});

db.createCollection("emprestimos", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: [
                "usuario_id",
                "livro_id",
                "dataEmprestimo",
                "dataPrevistaDevolucao",
                "status",
            ],
            properties: {
                usuario_id: { bsonType: "objectId" },
                livro_id: { bsonType: "objectId" },
                dataEmprestimo: { bsonType: "date" },
                dataPrevistaDevolucao: { bsonType: "date" },
                dataDevolucao: { bsonType: ["date", "null"] },
                status: { enum: ["emprestado", "devolvido"] },
            },
        },
    },
});

```

```

    },
  },
});

db.createCollection("reservas", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["usuario_id", "livro_id", "dataReserva",
"status"],
      properties: {
        usuario_id: { bsonType: "objectId" },
        livro_id: { bsonType: "objectId" },
        dataReserva: { bsonType: "date" },
        status: { enum: ["ativa", "finalizada"] },
      },
    },
  },
});

db.createCollection("avaliacoes", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["usuario_id", "livro_id", "nota", "comenta
rio", "data"],
      properties: {
        usuario_id: { bsonType: "objectId" },
        livro_id: { bsonType: "objectId" },
        nota: { bsonType: "number", minimum: 1, maximum: 5
},
        comentario: { bsonType: "string" },
        data: { bsonType: "date" },
      },
    },
  },
});

```

```
db.createCollection("logs", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["colecacao", "operacao", "data", "dados"],
      properties: {
        colecao: { bsonType: "string" },
        operacao: { bsonType: "string" },
        data: { bsonType: "date" },
        dados: { bsonType: "object" },
      },
    },
  },
});
```

Você pode ver o script completo no arquivo do projeto: `db/00-modelagem.js`.

Parte 2: Carga inicial

Usei o script colocado por você lá no moodle, fiz apenas um ajuste em relação aos ids dos usuários que estava dando um erro.

Você pode ver o script completo no arquivo do projeto: `db/01-carga.js`.

Parte 3: CRUD completo (backend)

Implementei as operações em endpoints REST no backend.

Você pode ver nas pastas do projeto: `backend/controllers/` e `backend/routes/`.

Abaixo uma tabela para exemplificar as rotas criadas:

Entidade	Operação	Rota
Livros	inserir	<code>POST /livros</code>
Livros	atualizar quantidade	<code>PATCH /livros/:id/quantidade</code>
Livros	alterar categoria	<code>PATCH /livros/:id/categoria</code>
Livros	remover	<code>DELETE /livros/:id</code>
Usuários	bloquear	<code>PATCH /usuarios/:id/bloquear</code>

Entidade	Operação	Rota
Usuários	reativar	PATCH /usuarios/:id/reativar
Usuários	alterar curso	PATCH /usuarios/:id/curso
Empréstimos	realizar	POST /emprestimos
Empréstimos	registrar devolução	PATCH /emprestimos/:id/devolucao
Empréstimos	renovar	PATCH /emprestimos/:id/renovar

Parte 4: Consultas intermediárias

Questão 1: Livros da categoria "Computação" publicados após 2020

```
db.livros.find({ categoria: "Computação", ano: { $gt: 2020 } });
```

Questão 2: Usuários cadastrados nos últimos 30 dias

```
const now = new Date();
const cutoff = new Date(now.getTime() - 30 * 24 * 60 * 60 * 1000);
db.usuarios.find({ dataCadastro: { $gte: cutoff, $lte: now } });
```

Questão 3: Empréstimos em atraso

```
db.emprestimos.find({
  status: "emprestado",
  dataPrevistaDevolucao: { $lt: new Date() },
});
```

Questão 4: Livros que nunca foram emprestados

```
db.livros.aggregate([
  {
    $lookup: {
```

```

        from: "emprestimos",
        localField: "_id",
        foreignField: "livro_id",
        as: "emprestimos",
    },
},
{ $match: { emprestimos: { $size: 0 } } },
{ $project: { titulo: 1, autor: 1, categoria: 1 } },
]);

```

Questão 5: Dez usuários com maior número de empréstimos

```

db.emprestimos.aggregate([
  { $group: { _id: "$usuario_id", totalEmprestimos: { $sum: 1 } } },
  { $sort: { totalEmprestimos: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "usuarios",
      localField: "_id",
      foreignField: "_id",
      as: "usuario",
    },
  },
  { $unwind: "$usuario" },
  {
    $project: {
      _id: 0,
      nome: "$usuario.nome",
      email: "$usuario.email",
      totalEmprestimos: 1,
    },
  },
]);

```

Parte 5: Aggregation pipeline avançado

Questão 6: Quantidade de livros por categoria

```
db.livros.aggregate([
  { $group: { _id: "$categoria", total: { $sum: 1 } } },
  { $sort: { total: -1 } },
]);
```

Questão 7: Média de avaliações por livro

```
db.avaliacoes.aggregate([
  {
    $group: {
      _id: "$livro_id",
      mediaNota: { $avg: "$nota" },
      totalAvaliacoes: { $sum: 1 },
    },
  },
  {
    $lookup: {
      from: "livros",
      localField: "_id",
      foreignField: "_id",
      as: "livro",
    },
  },
  { $unwind: "$livro" },
  {
    $project: {
      _id: 0,
      titulo: "$livro.titulo",
      mediaNota: { $round: ["$mediaNota", 2] },
      totalAvaliacoes: 1,
    },
  },
  { $sort: { mediaNota: -1 } },
]);
```

Questão 8: Dez livros mais emprestados

```
db.emprestimos.aggregate([
  { $group: { _id: "$livro_id", totalEmprestimos: { $sum: 1 } } },
  { $sort: { totalEmprestimos: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "livros",
      localField: "_id",
      foreignField: "_id",
      as: "livro",
    },
  },
  { $unwind: "$livro" },
  {
    $project: {
      _id: 0,
      titulo: "$livro.titulo",
      autor: "$livro.autor",
      totalEmprestimos: 1,
    },
  },
]);
```

Questão 9: Cursos que mais utilizam a biblioteca

```
db.emprestimos.aggregate([
  {
    $lookup: {
      from: "usuarios",
      localField: "usuario_id",
      foreignField: "_id",
      as: "usuario",
    },
  },
  { $unwind: "$usuario" },
```

```

    { $group: { _id: "$usuario.curso", totalEmprestimos: { $sum: 1 } } },
    { $sort: { totalEmprestimos: -1 } } },
  ]));

```

Questão 10: Taxa de devolução por mês

```

db.emprestimos.aggregate([
  {
    $group: {
      _id: {
        ano: { $year: "$dataEmprestimo" },
        mes: { $month: "$dataEmprestimo" },
      },
      total: { $sum: 1 },
      devolvidos: {
        $sum: { $cond: [{ $eq: ["$status", "devolvido"] },
1, 0] },
      },
    },
  },
  {
    $project: {
      _id: 0,
      ano: "$_id.ano",
      mes: "$_id.mes",
      total: 1,
      devolvidos: 1,
      taxaDevolucao: {
        $round: [
          { $multiply: [{ $divide: ["$devolvidos", "$total"] }, 100] },
          1,
        ],
      },
    },
  },
]);

```

```
    { $sort: { ano: 1, mes: 1 } } },  
  ] );
```

Questão 11: Ranking dos autores mais lidos

```
db.emprestimos.aggregate([  
  {  
    $lookup: {  
      from: "livros",  
      localField: "livro_id",  
      foreignField: "_id",  
      as: "livro",  
    },  
  },  
  { $unwind: "$livro" },  
  { $group: { _id: "$livro.autor", totalEmprestimos: { $sum: 1 } } },  
  { $sort: { totalEmprestimos: -1 } },  
  { $limit: 10 },  
]);
```

Questão 12: Livros com nota média inferior a 3

```
db.avaliacoes.aggregate([  
  {  
    $group: {  
      _id: "$livro_id",  
      mediaNota: { $avg: "$nota" },  
      totalAvaliacoes: { $sum: 1 },  
    },  
  },  
  { $match: { mediaNota: { $lt: 3 } } },  
  {  
    $lookup: {  
      from: "livros",  
      localField: "_id",  
      foreignField: "_id",  
    },  
  },  
]);
```

```

        as: "livro",
      },
    },
    { $unwind: "$livro" },
    {
      $project: {
        _id: 0,
        titulo: "$livro.titulo",
        mediaNota: { $round: ["$mediaNota", 2] },
        totalAvaliacoes: 1,
      },
    },
    { $sort: { mediaNota: 1 } } },
  ]);

```

Parte 6: Uso de **\$lookup**

Questão 13: Empréstimos com nome do usuário, título do livro e data

```

db.emprestimos.aggregate([
  {
    $lookup: {
      from: "usuarios",
      localField: "usuario_id",
      foreignField: "_id",
      as: "usuario",
    },
  },
  {
    $lookup: {
      from: "livros",
      localField: "livro_id",
      foreignField: "_id",
      as: "livro",
    },
  },
])

```

```

{ $unwind: "$usuario" },
{ $unwind: "$livro" },
{
  $project: {
    _id: 0,
    usuario: "$usuario.nome",
    livro: "$livro.titulo",
    dataEmprestimo: 1,
  },
},
]);

```

Questão 14: Avaliações com usuário, livro, nota e comentário

```

db.avaliacoes.aggregate([
  {
    $lookup: {
      from: "usuarios",
      localField: "usuario_id",
      foreignField: "_id",
      as: "usuario",
    },
  },
  {
    $lookup: {
      from: "livros",
      localField: "livro_id",
      foreignField: "_id",
      as: "livro",
    },
  },
  { $unwind: "$usuario" },
  { $unwind: "$livro" },
  {
    $project: {
      _id: 0,
      usuario: "$usuario.nome",
      livro: "$livro.titulo",

```

```

        nota: 1,
        comentario: 1,
    },
},
]);

```

Questão 15: Relatório completo de um usuário

```

db.usuarios.aggregate([
  { $match: { _id: ObjectId("6a381c88e3f1dcf0b99df8a3") } },
  {
    $lookup: {
      from: "emprestimos",
      localField: "_id",
      foreignField: "usuario_id",
      as: "emprestimos",
    },
  },
  {
    $lookup: {
      from: "reservas",
      localField: "_id",
      foreignField: "usuario_id",
      as: "reservas",
    },
  },
  {
    $lookup: {
      from: "avaliacoes",
      localField: "_id",
      foreignField: "usuario_id",
      as: "avaliacoes",
    },
  },
  {
    $project: {
      _id: 0,

```

```

    nome: 1,
    email: 1,
    curso: 1,
    cidade: 1,
    totalEmprestimos: { $size: "$emprestimos" },
    totalReservas: { $size: "$reservas" },
    totalAvaliacoes: { $size: "$avaliacoes" },
    emprestimos: 1,
    reservas: 1,
    avaliacoes: 1,
  },
},
]);

```

Parte 7: Índices

Criei os índices usando os seguintes comandos:

```

db.livros.createIndex({ titulo: 1 });
db.livros.createIndex({ autor: 1 });
db.livros.createIndex({ isbn: 1 }, { unique: true });
db.livros.createIndex({ categoria: 1 });
db.usuarios.createIndex({ email: 1 }, { unique: true });
db.usuarios.createIndex({ curso: 1 });

```

Comparei o desempenho com `explain("executionStats")` antes e depois, por exemplo:

```

db.livros.find({ titulo: "Livro 250" }).explain("executionStats");

```

Resultado observado:

Consulta	Sem índice (COLLSCAN)	Com índice (IXSCAN)
<code>livros.titulo = "Livro 250"</code>	500	1
<code>livros.autor = "Autor 50"</code>	500	4

Consulta	Sem índice (COLLSCAN)	Com índice (IXSCAN)
<code>livros.isbn = "97885100000250"</code>	500	1
<code>livros.categoria = "Computação"</code>	500	76
<code>usuarios.email = "usuario50@email.com"</code>	100	1
<code>usuarios.curso = "Engenharia de Software"</code>	100	24

Sem índice o plano usado foi `COLLSCAN` ele varreu toda a coleção. Já com o índice, o plano passa a ser o `IXSCAN`, ele examinou apenas os documentos que casam com a busca.

Obs: Notei que como o banco de dados tem um volume pequeno o tempo é praticamente igual, então a métrica relevante aqui é mais a redução de documentos examinados, o que em larga escala ficaria mais perceptível em relação ao tempo de execução.

Parte 8: Atlas Search

Criei o índice de busca criado pela interface do atlas na coleção `livros` com o nome `livro_palavra_chave`, mapeando apenas o campo `palavrasChave`:

[← Back to Index Overview](#)

Edit Search Index

livro_categoria **READY** QUERYABLE

This search index parses the data in **biblioteca.livros** and has the following configurations.

[View Atlas Search Docs](#)

```
1  {  
2    "mappings": {  
3      "dynamic": false,  
4      "fields": {  
5        "categoria": {  
6          "type": "string"  
7        }  
8      }  
9    }  
10 }
```

Cancel

Save

Questão 16: Pesquisar livros contendo "mongodb"

...

>

CLUSTER

>

COLLECTION

>

SEARCH INDEX

Cluster0

biblioteca.livros

livro_palavra_chave

[← Back to Search Indexes](#)

Search Tester

livro_palavra_chave READY QUERYABLE

Enter a term below to run a simple \$search query on these sample documents from **biblioteca.livros** using your search index and see the top 10 documents that are returned ranked by **score**. To edit \$search query or to run [\\$searchMeta](#), click "Edit Query" button. To run queries with stages after \$search, visit the [aggregation pipeline](#).

X

Search

Edit Query

This search took 0.01 seconds

SCORE: 0.0004538646899163723

_id: "6a381c89e3f1dcf0b99df907"

ano : 2015

autor : "Autor 29"

categoria : "Programação"

editora : "Novatec"

isbn : "9788510000001"

palavrasChave : Array (4)

0: "programação"

1: "algoritmos"

2: "mongodb"

3: "dados"

quantidade : 0

titulo : "Livro 1"

▼

SCORE: 0.0004538646899163723

_id: "6a381c89e3f1dcf0b99df908"

ano : 2000

autor : "Autor 45"

categoria : "Engenharia"

>

SCORE: 0.0004538646899163723

_id: "6a381c89e3f1dcf0b99df909"

ano : 2021

>

Questão 17: Pesquisar termos semelhantes a "algoritmos"

...

CLUSTER
Cluster0

>

COLLECTION
biblioteca.livros

>

SEARCH INDEX
livro_palavra_chave

[← Back to Search Indexes](#)

Search Tester

livro_palavra_chave
READY
QUERYABLE

Enter a term below to run a simple \$search query on these sample documents from **biblioteca.livros** using your search index and see the top 10 documents that are returned ranked by **score**. To edit \$search query or to run [\\$searchMeta](#), click "Edit Query" button. To run queries with stages after \$search, visit the [aggregation pipeline](#).

algoritmos

X

Search

Edit Query

This search took 0.01 seconds

SCORE: 0.0004538646899163723
_id: "6a381c89e3f1dcf0b99df907"

ano : 2015
autor : "Autor 29"
categoria : "Programação"
editora : "Novatec"
isbn : "9788510000001"
palavrasChave : Array (4)
0: "programação"
1: "algoritmos"
2: "mongodb"
3: "dados"
quantidade : 0
titulo : "Livro 1"

SCORE: 0.0004538646899163723
_id: "6a381c89e3f1dcf0b99df908"

ano : 2000
autor : "Autor 45"
categoria : "Engenharia"

SCORE: 0.0004538646899163723
_id: "6a381c89e3f1dcf0b99df909"

ano : 2021

Questão 18: Autocomplete para títulos

Criei o índice de busca pela interface do atlas na coleção `livros` com o nome `livro_titulo_autocomplete`, mapeando o campo `titulo` com o tipo `autocomplete` e `string`

...

>

CLUSTER
Cluster0 ▾

>

COLLECTION
biblioteca.livros ▾

>

SEARCH INDEX
livro_titulo_autocomplete ▾

[← Back to Index Overview](#)

Edit Search Index

livro_titulo_autocomplete READY QUERYABLE

This search index parses the data in **biblioteca.livros** and has the following configurations.

[View Atlas Search Docs](#)

```
1  {
2    "mappings": {
3      "dynamic": false,
4      "fields": {
5        "titulo": [
6          {
7            "type": "autocomplete"
8          },
9          {
10           "type": "string"
11         }
12       ]
13     }
14   }
15 }
```

Cancel

Save

Resultados buscando por "livro 2":

Cluster0 ▾ biblioteca.livros ▾ livro_titulo_autocomplete ▾

Reset Create Query From Template Search

```

1 [
2   {
3     $search: {
4       index: "livro_titulo_autocomplete",
5       autocomplete: { query: "livro 2", path: "titulo" }
6     }
7   }
8 ]

```

[Learn about creating and running queries](#)

This search took 0.03 seconds

SCORE: 7.748305797576904 _id: "6a381c89e3f1dcf0b99df908"

ano : 2000
autor : "Autor 45"
categoria : "Engenharia"
editora : "Campus"
isbn : "97885100000002"
palavrasChave : Array (4)
quantidade : 3
titulo : "Livro 2"

SCORE: 1.050205111503601 _id: "6a381c89e3f1dcf0b99df91a"

ano : 2003
autor : "Autor 45"
categoria : "Banco de Dados"
editora : "Elsevier"
isbn : "97885100000020"
palavrasChave : Array (4)
quantidade : 13
titulo : "Livro 20"

Parte 9: Atlas Triggers

Trigger 1: Registrar em logs toda inserção de empréstimo

Trigger criado na coleção `emprestimos` usando o evento `Insert` e a seguinte função:

```

exports = async function (changeEvent) {
  const db = context.services.get("Cluster0").db("biblioteca");
  const doc = changeEvent.fullDocument;
  await db.collection("logs").insertOne({
    colecao: "emprestimos",
    operacao: "insert",
  });
}

```

```

data: new Date(),
dados: {
  emprestimo_id: doc._id,
  usuario_id: doc.usuario_id,
  livro_id: doc.livro_id,
},
});
};

```

Abaixo o print:

ORGANIZATION: Guilherme's Org - 20... PROJECT: projeto-tbd TRIGGER: insercao_emprestimo

← Back to Triggers

Edit Trigger

Enable ☒

Trigger Details Database Trigger (Watch Against "emprestimos" Collection) ▼

Trigger Configurations EVENT ORDERING ▼

Event Type Function ^

Select An Event Type

[Learn how to use Amazon EventBridge with Triggers](#)

Function EventBridge

Function

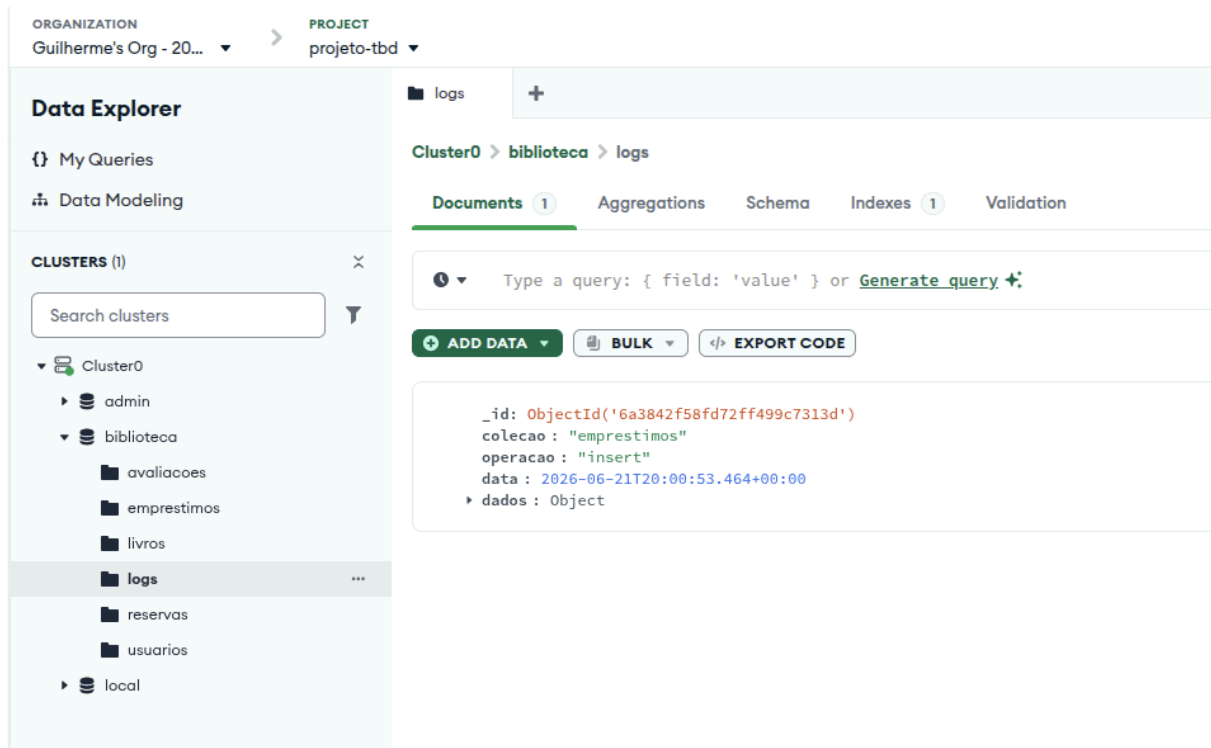
Add Dependency Function Snippets

```

1 exports = async function (changeEvent) {
2   const db = context.services.get("Cluster0").db("biblioteca");
3   const doc = changeEvent.fullDocument;
4   await db.collection("logs").insertOne({
5     colecao: "emprestimos",
6     operacao: "insert",
7     data: new Date(),
8     dados: {
9       emprestimo_id: doc._id,
10      usuario_id: doc.usuario_id,
11      livro_id: doc.livro_id,
12    },
13  });
14 };

```

Logs sendo computados:



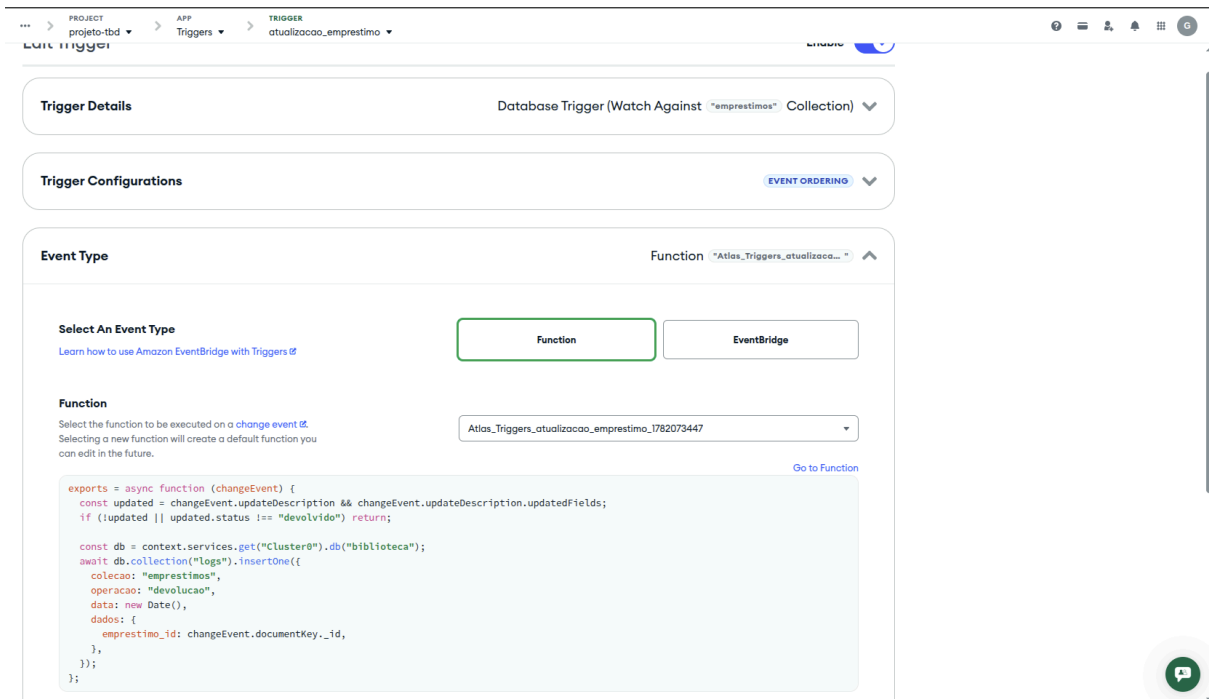
Trigger 2: Registrar devoluções

Trigger criado na coleção `emprestimos` usando o evento de `Update` e a seguinte função:

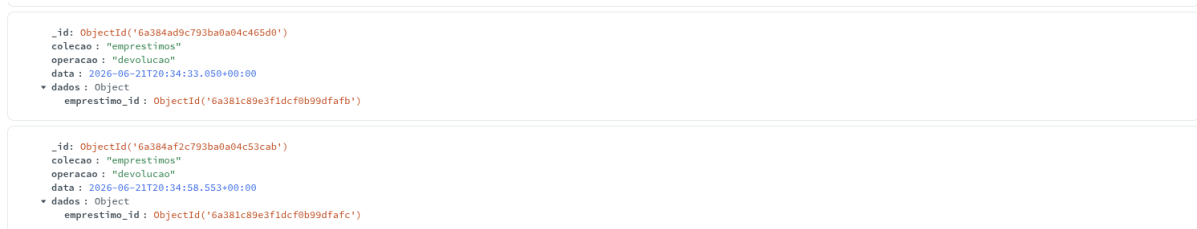
```
exports = async function (changeEvent) {
  const updated = changeEvent.updateDescription && changeEvent.updateDescription.updatedFields;
  if (!updated || updated.status !== "devolvido") return;

  const db = context.services.get("Cluster0").db("biblioteca");
  await db.collection("logs").insertOne({
    colecao: "emprestimos",
    operacao: "devolucao",
    data: new Date(),
    dados: {
      emprestimo_id: changeEvent.documentKey._id,
    },
  });
};
```

Abaixo o print:



Logs sendo computados:



Trigger 3: Notificar quando um livro fica sem exemplares

Coleção **livros**, evento **Update**. Function:

```
exports = async function (changeEvent) {
  const doc = changeEvent.fullDocument;
  if (!doc || doc.quantidade > 0) return;
  const db = context.services.get('Cluster0').db('biblioteca');
  await db.collection('logs').insertOne({
    colecao: 'livros',
    operacao: 'sem_estoque',
    data: new Date(),
    dados: { livro_id: doc._id, titulo: doc.titulo },
  });
}
```

```
});  
};
```

Abaixo o print:

Edit Trigger Enable ✓

Trigger Details Database Trigger (Watch Against "livros" Collection) ▼

Trigger Configurations EVENT ORDERING ▼

Event Type Function "Atlas_Triggers_notificacao..." ▲

Select An Event Type
[Learn how to use Amazon EventBridge with Triggers](#)

Function
Select the function to be executed on a [change event](#).
Selecting a new function will create a default function you can edit in the future.

Atlas_Triggers_notificacao_sem_estoque ▼

[Go to Function](#)

```
exports = async function (changeEvent) {  
  const updated = changeEvent.updateDescription && changeEvent.updateDescription.updatedFields;  
  if (!updated || updated.quantidade == null || updated.quantidade > 0) return;  
  
  const db = context.services.get("Cluster0").db("biblioteca");  
  await db.collection("logs").insertOne({  
    colecao: "livros",  
    operacao: "sem_estoque",  
    data: new Date(),  
    dados: { livro_id: changeEvent.documentKey._id },  
  });  
};
```

Logs sendo computados:

Data Explorer

My Queries
Data Modeling

CLUSTERS (1)
Cluster0
admin
biblioteca
avaliacoes
emprestimos
livros
logs
reservas
usuarios
local

Cluster0 > biblioteca > logs

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [BULK](#) [EXPORT CODE](#)

25 1 - 6 of 6 < > ⌵ ⌵ ⌵

<pre>{ "_id": ObjectId("6a384987c793ba8a4b91ad8"), "colecao": "emprestimos", "operacao": "devolucao", "data": 2026-06-21T29:28:55.736+00:00, "dados": Object }</pre>
<pre>{ "_id": ObjectId("6a384a9c793ba8a4c465d8"), "colecao": "emprestimos", "operacao": "devolucao", "data": 2026-06-21T29:34:33.050+00:00, "dados": Object }</pre>
<pre>{ "_id": ObjectId("6a384af2c793ba8a4c53cab"), "colecao": "emprestimos", "operacao": "devolucao", "data": 2026-06-21T29:34:58.553+00:00, "dados": Object }</pre>
<pre>{ "_id": ObjectId("6a384cac5f5984ef4f6693d4"), "colecao": "livros", "operacao": "sem_estoque", "data": 2026-06-21T29:42:20.660+00:00, "dados": { "livro_id": ObjectId("6a381c89e3f1dcfb99df907") } }</pre>

Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Parte 10: MongoDB Charts

Gráfico 1: Livros por categoria

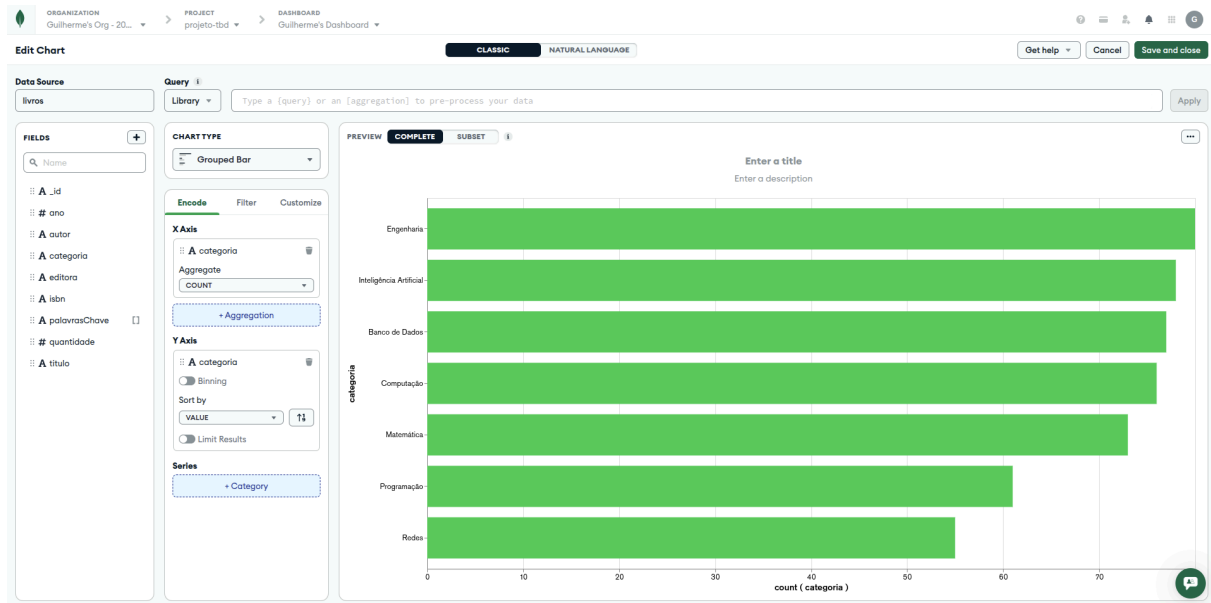


Gráfico 2: Empréstimos por mês

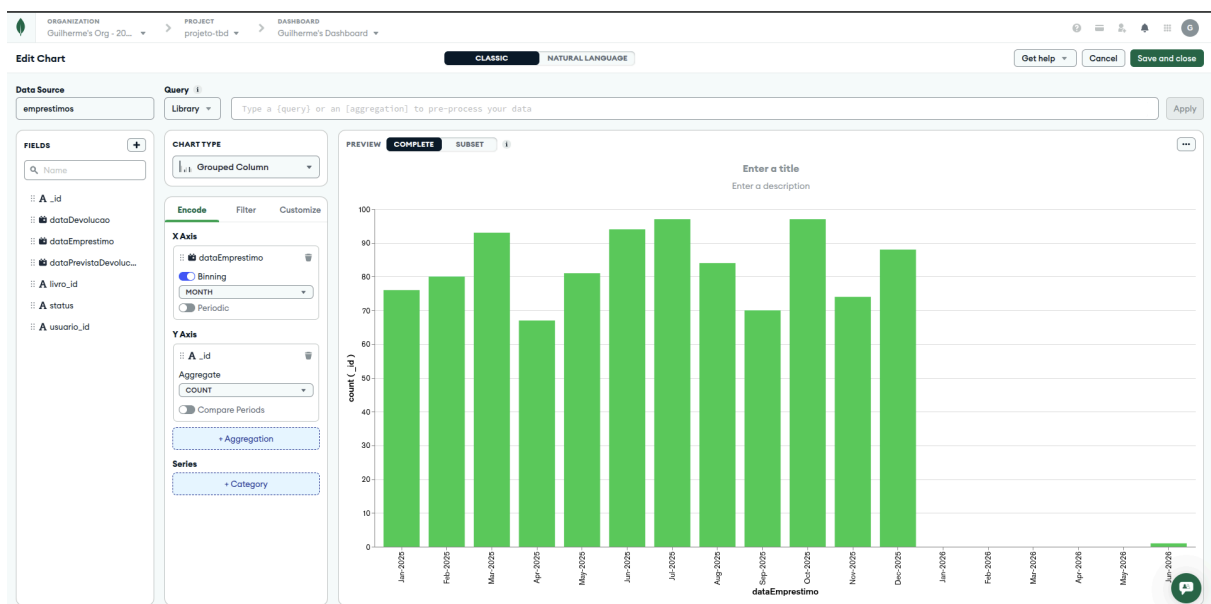


Gráfico 3: Top 10 livros

Aqui eu usei um lookup com livros e limitei/ordenei os resultados

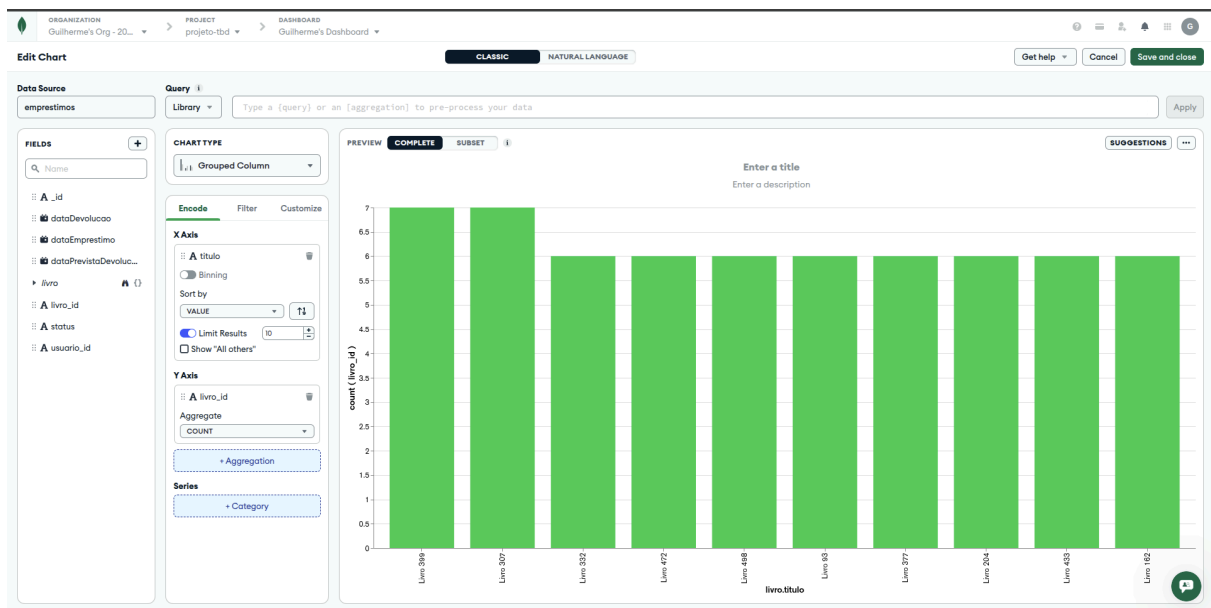
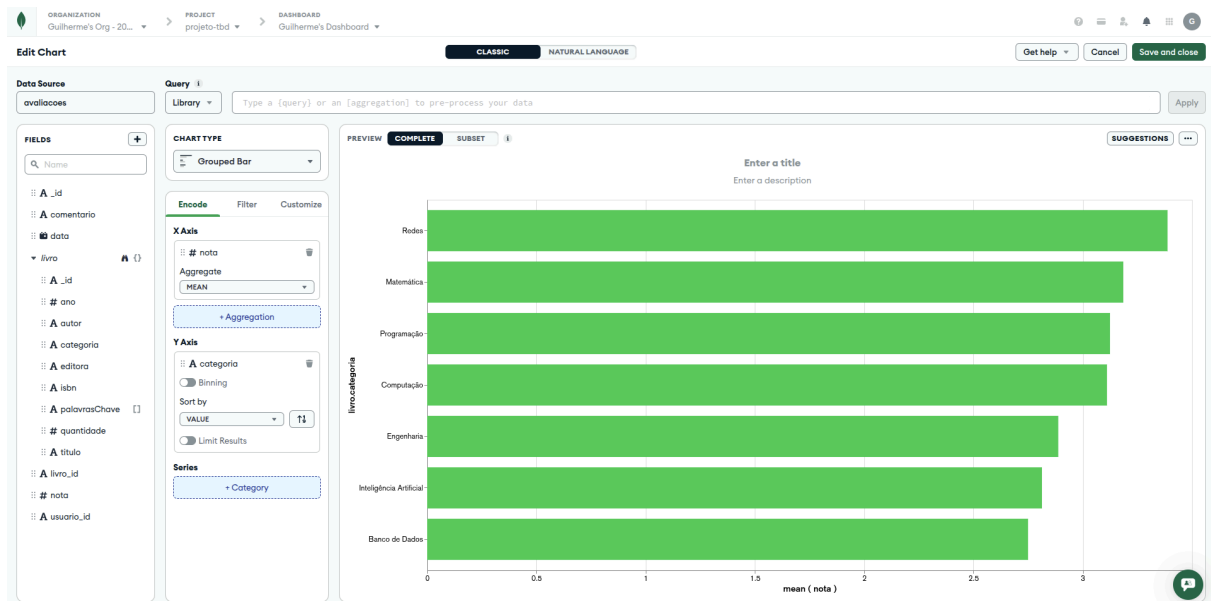
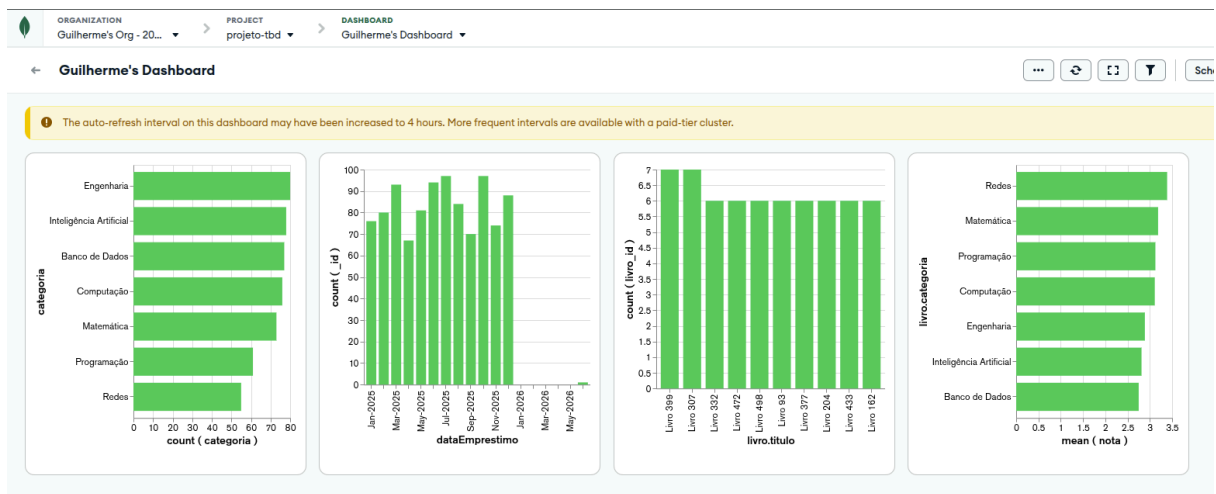


Gráfico 4: Avaliação média por categoria

Também usei o lookup em livros e utilizei o `mean` para fazer a média:



Visão final com todos os gráficos:



Parte 12: Transações

Atualizei a criação de empréstimo para ser executada em uma transação com três operações: inserir o empréstimo, diminuir a quantidade do livro e registrar o log.

O código está em: `backend/services/loans.service.js`.

Obs: pode ocorrer duplicação de logs, visto que já criei um trigger na inserção de empréstimos.

Parte 14: Modelagem e escalabilidade

1. Quais coleções devem ser shardadas?

Acredito que as principais candidatas de início seriam `emprestimos`, `reservas` e `avaliacoes`.

2. Qual shard key seria utilizada?

- Para `emprestimos` usaria a key de `usuario_id` com `hashed`.
- Para `reservas` eu seguiria a mesma lógica de `emprestimos`.
- Já para `avaliacoes` eu faria uma chave composta entre `livro_id` e `_id`.

3. Justificativas

- Em `emprestimos` priorizei o `usuario_id` porque imagino que em um sistema todos os usuários farão a query de ver seus empréstimos, mas poderia ser por `livro_id` também, caso a aplicação tenha telas que fazem com que o

livro seja o principal. Usaria `hashed` na opção por `usuario_id` porque o volume seria menor. Se fosse por `livro_id`, trocaria para uma chave composta `{ livro_id, _id }`, porque o volume pode aumentar e seria interessante dividir um livro muito popular entre shards.

- Em `reservas` seguiria a mesma lógica de `emprestimos` olhando para `usuario_id`.
- E em `avaliacoes` usaria o `livro_id` para usar o livro como base e o `_id` para dar granularidade e dividir avaliações por exemplo de um livro muito popular em vários shards.

4. Como o crescimento para 10 milhões de empréstimos afetaria o projeto?

Com esse volume, acredito que índices e sharding deixam de ser opcionais e passam a ser indispensáveis.

Sem índices cada consulta precisaria varrer a coleção inteira, e o tempo de resposta cresceria junto com a quantidade de empréstimos, ficando cada vez mais lento.

Por isso eu criaria índices nos campos que as telas mais consultam em empréstimos:

- `{ usuario_id: 1, dataEmprestimo: -1 }`: para o histórico do usuário, já trazendo os empréstimos ordenados por data.
- `{ status: 1, dataPrevistaDevolucao: 1 }`: para listar empréstimos ativos e atrasados.
- `{ livro_id: 1 }`: para consultas por livro e para o ranking dos mais emprestados.

E com a shard key definida na questão anterior, acho que a leitura e a escrita vão ficar bem distribuídas entre os shards, mantendo o desempenho de forma horizontal e escalável.

Parte 15: Auditoria e segurança

Questão 19: Usuários com permissões diferentes

Tentei executar os comandos abaixo mas infelizmente o atlas não deixa fazer isso via Compass na versão gratuita:

```
db.createUser({
  user: "admin_biblioteca",
  pwd: "trocar_senha",
  roles: [{ role: "dbAdmin", db: "biblioteca" }]
});

db.createUser({
  user: "bibliotecario",
  pwd: "trocar_senha",
  roles: [{ role: "readWrite", db: "biblioteca" }]
});

db.createUser({
  user: "consulta",
  pwd: "trocar_senha",
  roles: [{ role: "read", db: "biblioteca" }]
});
```

Sendo assim, tive que criar os usuários direto pelo front mesmo, abaixo o

`admin`:

MongoDB uses [SCRAM](#) as its default authentication method.

Password Authentication

admin

Edit Password

User Description

Add an optional description to your user.

Database User Privileges

Configure role based access control by assigning database user a mix of one built-in role, multiple custom roles, and multiple specific privileges. A user will gain access to all actions within the roles assigned to them, not just the actions those roles share in common. **You must choose at least one role or privilege.** [Learn more about roles.](#)

Built-in Role

Select one [built-in role](#) for this user.

Custom Roles

Select your [pre-defined custom role\(s\)](#). Create a custom role in the [Custom Roles](#) [tab](#).

Specific Privileges

1 SELECTED

Select multiple privileges and what database and collection they are associated with. Leaving collection blank will grant this role for all collections in the database.

dbAdmin

@

biblioteca

Collection*

Add Specific Privilege

Restrict Access to Specific Clusters/Federated Database Instances/Stream Processing

Agora o **bibliotecario** :

MongoDB uses [SCRAM](#) as its default authentication method.

Password Authentication

bibliotecario

Edit Password

User Description

Add an optional description to your user.

Database User Privileges

Configure role based access control by assigning database user a mix of one built-in role, multiple custom roles, and multiple specific privileges. A user will gain access to all actions within the roles assigned to them, not just the actions those roles share in common. **You must choose at least one role or privilege.** [Learn more about roles.](#)

Built-in Role

Select one [built-in role](#) for this user.

Custom Roles

Select your [pre-defined custom role\(s\)](#). Create a custom role in the [Custom Roles](#) [tab](#).

Specific Privileges

1 SELECTED

Select multiple privileges and what database and collection they are associated with. Leaving collection blank will grant this role for all collections in the database.

readWrite

@

biblioteca

Collection*

Add Specific Privilege

E por último o **consultor**:

MongoDB uses [SCRAM](#) as its default authentication method.

Password Authentication

consultor

Edit Password

User Description

Add an optional description to your user.

Database User Privileges

Configure role based access control by assigning database user a mix of one built-in role, multiple custom roles, and multiple specific privileges. A user will gain access to all actions within the roles assigned to them, not just the actions those roles share in common. **You must choose at least one role or privilege.** [Learn more about roles.](#)

Built-in Role

Select one [built-in role](#) for this user.

Custom Roles

Select your [pre-defined custom role\(s\)](#). Create a custom role in the [Custom Roles](#) tab.

Specific Privileges

Select multiple privileges and what database and collection they are associated with. Leaving collection blank will grant this role for all collections in the database.

1 SELECTED

read

@

biblioteca

Collection*

Add Specific Privilege

Ficando assim:

Database Users

+ ADD NEW DATABASE USER					
User	Description	Authentication Method	MongoDB Roles	Resources	Actions
admin		SCRAM	dbAdmin@biblioteca	All Resources	EDIT DELETE
bibliotecario		SCRAM	readWrite@biblioteca	All Resources	EDIT DELETE
consultor		SCRAM	read@biblioteca	All Resources	EDIT DELETE
guiwb		SCRAM	atlasAdmin@admin	All Resources	EDIT DELETE

Questão 20: IP Access List

Meu IP já veio por padrão no access list, não adicionei mais nenhum porque não senti a necessidade:

+ADD IP ADDRESS			
You will only be able to connect to your cluster from the following list of IP Addresses:			
IP Address	Comment	Status	Actions
177.22.171.30/32 (includes your current IP address)	Created as part of the Auto Setup process	Active	EDIT DELETE

Questão 21: Auditoria pelos logs da aplicação

Ativei ela lá no Atlas:

Database Auditing

Database auditing allows you to customize log downloads with the users, groups, and actions you want to audit.

Turning on this feature will increase your daily cluster pricing. [Read more.](#)



Parte 16: Projeto integrado

Foi feita a aplicação web completa.

Backend (`backend/`): usei o Node.js + Express + driver mongodb com CRUD de livros, usuários, empréstimos, reservas, avaliações e criei novos endpoints de relatório (`/relatorios/dashboard` , `/relatorios/livros-populares` , `/relatorios/usuario/:id`).

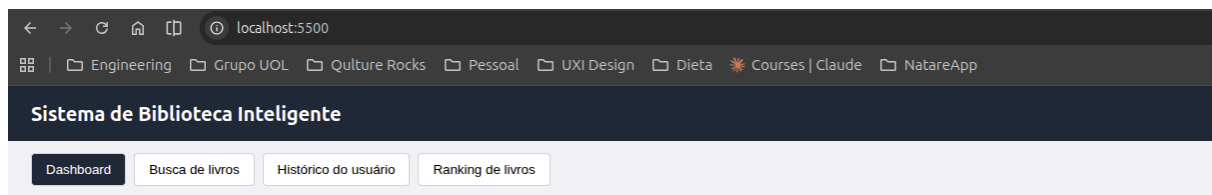
Frontend (`frontend/`): usei HTML + JS puro, com dashboard administrativo, busca de livros, histórico do usuário e ranking dos livros mais populares. Consumo da API isolado em

`frontend/services/api.js` .

Para rodar o back e o front execute os seguintes comandos de forma separada:

```
cd backend && npm install && npm start           # API em ht
tp://localhost:3000
cd frontend && python3 -m http.server 5500         # site em
http://localhost:5500
```

A API seria exposta em `http://localhost:3000` e o frontend em `http://localhost:5500` , abaixo um print de como ficou o front:



Dashboard administrativo

Usuários: 100

Livros: 500

Empréstimos: 1000

Reservas: 300

Avaliações: 1000

Livros por categoria: Engenharia: 80 · Inteligência Artificial: 78 · Banco de Dados: 77 · Computação: 76 · Matemática: 73 · Programação: 61 · Redes: 55